Bogazici University

CmpE 48A – Cloud Computing

Term Project Technical Report

Mustafa Batuhan Büber
2021400231

Kerem Haktan Kurt
2021400150

Links

Demo video: YouTube
Repository: GitHub

April 9, 2026

Contents

1 Cloud Architecture **2**

2 Deployment Process **3**
2.1 Terraform File Structure 3
2.2 Workflow 4

3 Locust Tests **4**
3.1 Configuration (`terraform.tfvars`) 5
3.2 Test Execution Scripts 5
3.2.1 User Behavior (`locustfile.py`) 5
3.2.2 Test Orchestrator & Monitoring (`run_hpa.py`) 6

4 Test Analysis & Observations **6**
4.1 Backend Scaling Sensitivity 7
4.1.1 Base Test 7
4.1.2 Decrease Backend CPU Limit 7
4.1.3 Increase Backend Max Replica 8
4.1.4 Increase Backend CPU Limit 8
4.2 Database & Compute Bottlenecks 9
4.2.1 Undersized Database VM (e2-medium) 9
4.2.2 Oversized Database, Undersized Cluster 10
4.2.3 Cluster Autoscaling & Connection Exhaustion 12
4.3 Connection Pool Optimization 14
4.4 Node Pre-Provisioning: Cold vs Warm Start 16

5 Cost Analysis **19**

1 Cloud Architecture

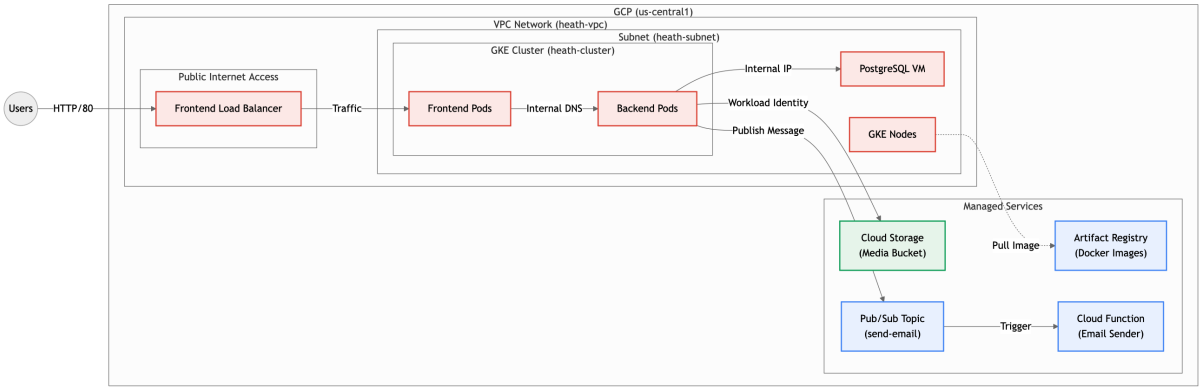

Figure 1: Cloud architecture diagram.

The application architecture follows a cloud-native approach deployed on Google Cloud Platform. The infrastructure is provisioned using Terraform and orchestrated by Google Kubernetes Engine (GKE). The system consists of the following components:

- **Virtual Private Cloud (VPC):** A custom VPC network isolates the infrastructure. It includes a dedicated subnet with secondary IP ranges for GKE Pods and Services, enabling VPC-native traffic routing. Firewall rules secure internal communication between nodes, pods, and virtual machines.
- **Google Kubernetes Engine (GKE):** The core application runs on a zonal GKE cluster (`heath-cluster`). It utilizes a separately managed node pool with autoscaling enabled. The cluster is configured with Workload Identity to securely authenticate Kubernetes workloads with Google Cloud services without managing static keys.
- **Frontend Service:** The frontend application is deployed as a Kubernetes Deployment with a Horizontal Pod Autoscaler (HPA). It is exposed via a LoadBalancer Service utilizing a reserved static external IP address, serving as the entry point for user traffic.
- **Backend Service:** The backend is deployed as a Kubernetes Deployment, also scaled via HPA. It runs with a dedicated Kubernetes Service Account mapped to a Google Service Account (Workload Identity), making sure it has the right permissions for Cloud Storage and Pub/Sub. It is exposed internally via a ClusterIP service, accessible only by the frontend.
- **Artifact Registry:** A Docker repository (`heath-repo`) stores the container images for both the frontend and backend applications, which are pulled by GKE during deployment.
- **Database (PostgreSQL VM):** A self-managed PostgreSQL database runs on a dedicated Compute Engine instance (`heath-postgres-vm`). The VM is configured with a startup script to install PostgreSQL, configure users, and allow connections solely from the VPC subnet and GKE Pod IP ranges.

- **Cloud Storage (GCS):** A generic Cloud Storage bucket (`*-media`) is used for storing user-generated content, such as profile photos. The backend service interacts with this bucket directly using its IAM credentials.
- **Serverless:**
  - **Pub/Sub:** A topic named `send-email` decouples the email sending process from the main application flow. The backend publishes messages to this topic when email notifications are required.
  - **Cloud Functions:** A 2nd Gen Cloud Function (`email-sender-function`) is triggered by messages on the Pub/Sub topic. It executes Java code to send emails via the SendGrid API, ensuring the main application remains responsive.

2 Deployment Process

The deployment is managed entirely through Terraform, using a modular file structure.

2.1 Terraform File Structure

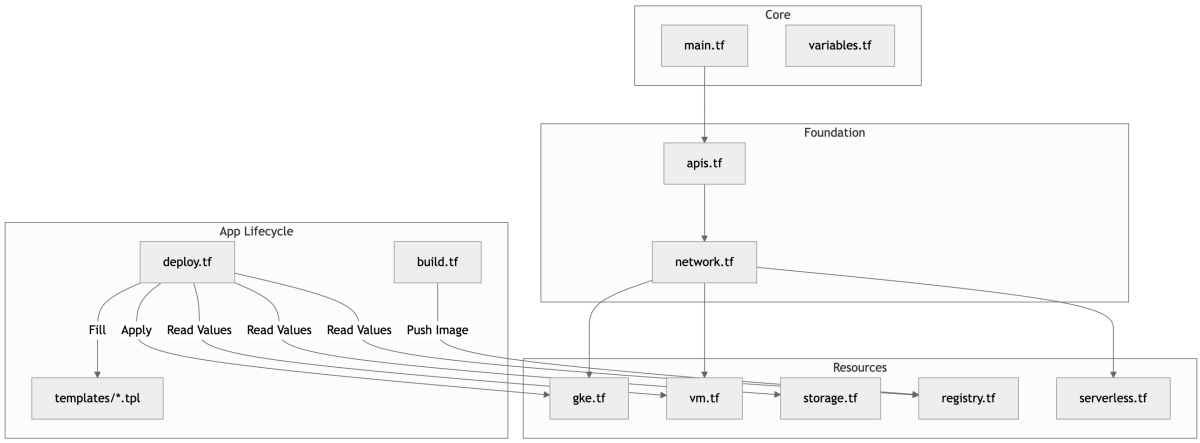

Figure 2: Terraform file structure.

- **Core Infrastructure:** `network.tf` defines the VPC, subnets, and firewalls, while `apis.tf` enables necessary Google Cloud APIs.
- **Compute & Database:** `gke.tf` provisions the Kubernetes cluster and node pools. `vm.tf` manages the PostgreSQL virtual machine instance.
- **Storage & Registry:** `storage.tf` creates GCS buckets, and `registry.tf` sets up the Artifact Registry for Docker images.
- **Serverless:** `serverless.tf` handles Pub/Sub topics and Cloud Functions.

- **Build & Deploy:** `build.tf` manages Cloud Build triggers for container images, and `deploy.tf` handles the generation of Kubernetes manifests and final application deployment.

2.2 Workflow

The automated deployment workflow follows these steps:

1. **Infrastructure Provisioning:** Terraform first provisions the foundational Google Cloud resources, including the VPC network, GKE cluster, Artifact Registry, Cloud Storage buckets, and the Compute Engine instance for the database.

2. **Database Initialization:** Upon creation of the PostgreSQL VM, a startup script automatically installs the database software, configures network listeners, updates firewall rules to allow traffic from the GKE subnet, and creates the necessary database users and schemas.

3. **Container Build & Push:** Terraform monitors the application source code for changes. When modifications are detected, a `null_resource` with a `local-exec` provisioner triggers Google Cloud Build (`gcloud builds submit`). This service builds the Docker images for the frontend and backend and pushes them to the private Artifact Registry.

4. **Manifest Generation:** Using the `templatefile` function, Terraform dynamically generates Kubernetes manifests from templates. It writes specific values such as the database internal IP, Service Account emails, and image repository paths into the YAML configuration files, which are saved locally to the `k8s/generated/` directory.

5. **Application Deployment:** Once the cluster is ready and images are built, a final Terraform resource executes a shell script to deploy the application. This script:

   - Authenticates `kubectl` with the GKE cluster.
   - Applies the generated manifests (`kubectl apply`).
   - Triggers a rollout restart (`kubectl rollout restart`) to ensure the pods pull the latest container images from the registry.

3 Locust Tests

To validate the architecture's scalability and resilience, we performed load tests using Locust to stress the **Horizontal Pod Autoscaler (HPA)** and **Cluster Autoscaler**. The tests evaluated dynamic scaling under traffic spikes, while also analyzing the impact of different **VM configurations** and monitoring for **database connection pool exhaustion** to ensure system stability.

- **Autoscaling Trigger Points:** How quickly the HPA detects CPU spikes and requests new replicas, and how fast the Cluster Autoscaler provisions new nodes.

- **Resource Bottlenecks:** Whether the system scales fast enough to prevent request queuing (the "Gap of Death") or if database connections become the limiting factor.

- **Recovery Time:** How the system stabilizes after scaling events occur.

3.1 Configuration (`terraform.tfvars`)

The system's behavior is controlled by a central Terraform configuration file. Table 1 details the wide range of variables we tested to find the optimal configuration. We experimented with different machine types, scaling limits, and user loads to simulate various production scenarios.

Category	Variable	Tested Values
Load Parameters	`USERS`	10 / 500 / 1000 / 2000
	`SPAWN_RATE`	1 / 20 / 50 (users/sec)
Database	`db_machine_type`	e2-medium / e2-standard-8 / e2-custom-6-24576
	`postgresql_max_connections`	100 / 1000
	`db_disk_size`	20 GB
GKE Cluster	`gke_node_machine_type`	e2-standard-2 / e2-standard-4
	`gke_autoscaling_enabled`	true / false
	`gke_min_node_count`	1
	`gke_max_node_count`	1 / 3
Backend	`backend_cpu_request`	250m / 500m
	`backend_cpu_limit`	500m / 1000m / 3000m
	`backend_memory_limit`	1Gi
	`backend_hpa_min_replicas`	1
	`backend_hpa_max_replicas`	3 / 9
	`backend_hpa_cpu_target`	70%
	`backend_db_pool_size`	10 / 50

Table 1: Infrastructure variables configured in `terraform.tfvars`, showing the full range of values tested during the experiment.

3.2 Test Execution Scripts

The load testing framework consists of two main Python scripts: `locustfile.py`, which defines the user behavior, and `run_hpa.py`, which orchestrates the test execution and metrics collection.

3.2.1 User Behavior (`locustfile.py`)

The `locustfile.py` defines the logic for simulated users. We utilized the **AuthenticatedUser** class to simulate a standard active user, creating a mix of CPU-intensive and Database-intensive operations. The user lifecycle interacts with the following endpoints:

1. **Authentication (High CPU):** Upon starting, the user performs CPU-intensive cryptographic operations:
   - `POST /api/auth/register`: Registers a new user account (BCrypt hashing).
   - `POST /api/auth/login`: Authenticates and retrieves a JWT token.
2. **Onboarding:** Immediately after login, the user submits personal details:
   - `POST /api/interest-form/submit`: Submits user profile data (Height, Weight, Gender, etc.).

3. **Workload Loop:** The user then loops through a set of weighted tasks:
   - **Feed (Read):** `GET /api/recipe/get-all` - Initial fetch to populate local state.
   - **Complex Read:** `GET /api/recipe/get-all` - Fetches the full recipe feed. This places high read load on the database.
   - **Single Read:** `GET /api/recipe/get` - Views details of a specific recipe ID.
   - **Write:** `POST /api/recipe/create` - Creates a new recipe. This tests database write performance and transactional integrity.

3.2.2 Test Orchestrator & Monitoring (`run_hpa.py`)

This script automates the execution of the Locust test and runs a parallel background thread that queries the Kubernetes cluster and Database logs in real-time. It captures the following specific metrics to validate our scaling criteria:

- **HPA Metrics:** It queries `kubectl get hpa` to track the `TARGET%` vs `CURRENT%` CPU utilization. This confirms if the autoscaler reacts correctly when the 70% threshold is breached.
- **Pod Metrics:** It parses `kubectl top pods` to record the exact CPU usage (in millicores) of every individual pod. This verifies if the `backend_cpu_limit` is being respected or throttled.
- **Deployment Replicas:** It tracks `Desired` vs. `Ready` replicas. A gap between these two metrics often indicates that pending pods are waiting for new nodes to be provisioned.
- **Node Scaling Count:** It monitors `kubectl get nodes` to track the Cluster Autoscaler adding or removing VM instances (scaling from 1 to 3 nodes).
- **Node CPU Utilization (%):** It executes `kubectl top nodes` to record the CPU percentage usage of each individual node VM. This helps us understand if the underlying infrastructure is fully utilized or if resources are being wasted.
- **DB Connection Pool:** By parsing the backend logs for `HikariCP` statistics, the script monitors the internal state of the connection pool:
  - **Idle:** Connections waiting in the pool.
  - **Waiting:** Requests blocked because the pool is exhausted.

  This metric was crucial for identifying bottlenecks, leading to the optimization of pool sizes and connection limits.

4 Test Analysis & Observations

This section details the specific behaviors observed during our load testing scenarios, focusing on system responsiveness, scaling latency, and resource utilization.

4.1 Backend Scaling Sensitivity

4.1.1 Base Test

Load	
Parameter	**Value**
USERS	10
SPAWN_RATE	1

Backend	
Parameter	**Value**
backend_cpu_request	250m
backend_cpu_limit	1000m
backend_hpa_enabled	True
backend_hpa_min_replicas	1
backend_hpa_max_replicas	3
backend_hpa_cpu_target	70%

GKE	
Parameter	**Value**
gke_node_machine_type	4 vCPU (4000m)
gke_node_count	1
gke_autoscaling_enabled	False

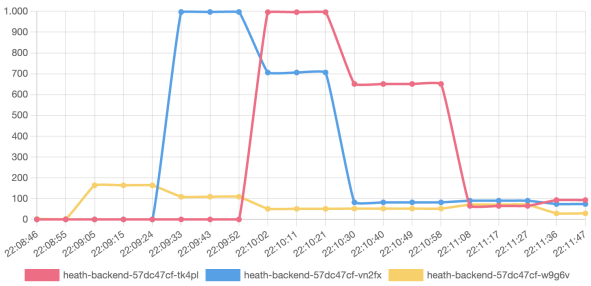

Figure 3: Backend pods CPU usage.

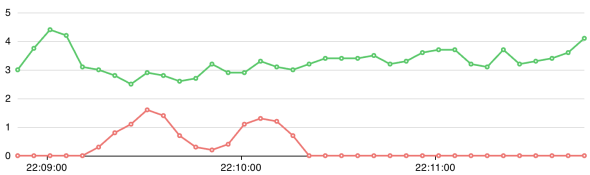

Figure 4: Locust RPS and failure rate.

At the start of the test, the incoming traffic pushes the backend CPU usage above 70% of the guaranteed `backend_cpu_request` (250m). Because of this, HPA requests a second backend replica (see Figure 3). Even though the new pod can use close to the CPU limit (1000m), a Java Spring backend needs some time to fully start up (19 secs). While it is still warming up, requests keep coming and failures increase for a short time (Figure 4), so HPA requests a third replica as well (Figure 3). That pod also needs a short warm-up period before it is ready. During these warm-up periods, we keep seeing failures, but once the new replicas become ready, the failures drop to zero and the system runs in a stable, steady state (Figure 4).

4.1.2 Decrease Backend CPU Limit

Parameter	Change
backend_cpu_limit	$1000m \rightarrow 500m$

To see how CPU availability affects the Java Spring backend warm-up time, we decreased `backend_cpu_limit`. With the lower CPU limit, the startup/warm-up time increased from 19 seconds to around 76 seconds (see Figure 5). Because new replicas take much longer to become ready, the system needs more time to reach a steady state, and we observe more failures during that period (see Figure 6).

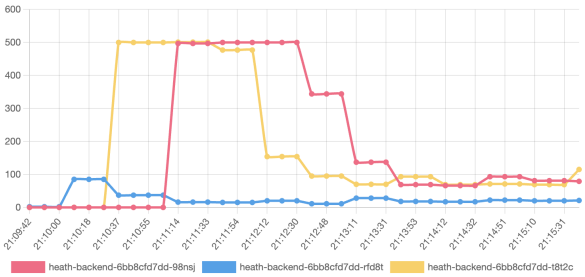

Figure 5: Backend pods CPU usage.

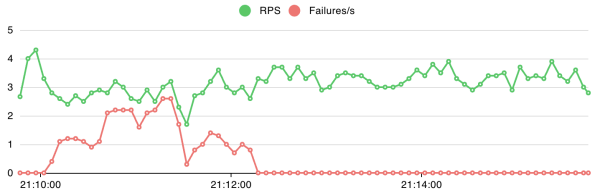

Figure 6: Locust RPS and failure rate.

4.1.3 Increase Backend Max Replica

Parameter	Change
backend_hpa_max_replicas	$3 \rightarrow 9$

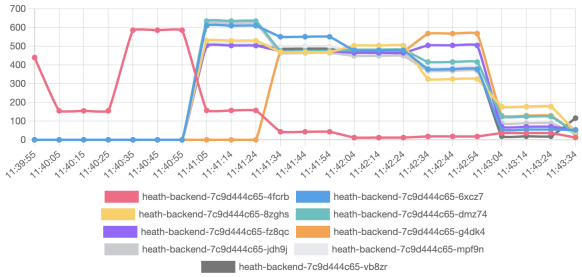

Figure 7: Backend pods CPU usage.

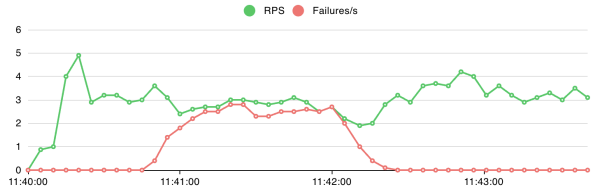

Figure 8: Locust RPS and failure rate.

In the previous test, we saw that decreasing `backend_cpu_limit` backfired. This time, we wanted to check whether increasing `backend_hpa_max_replicas` would help, so we raised it from 3 to 9. When the load hit at the beginning of the test, HPA did request new replicas, but it requested almost all of them at once (see Figure 7). During the Java Spring startup phase, the pods could not even reach the 1000m CPU limit, because there were 9 replicas sharing the node's CPU capacity. As a result, each replica was only able to use around 500–600m CPU (see Figure 7). In practice, this looked similar to running with a lower CPU limit (like the previous test), so the system again took longer to stabilize and we still observed failures during the warm-up period (see Figure 8). In short, increasing the max replica count did not help in this setup.

4.1.4 Increase Backend CPU Limit

Parameter	Change
backend_cpu_limit	$500m \rightarrow 3000m$

In the previous tests, we noticed that we could not fully avoid failures during the Java Spring backend warm-up period after new replicas start. This time, we tried setting `backend_cpu_limit` to a much higher value (3000m). Since `backend_cpu_request` stayed at 250m, we expected the CPU utilization to pass the HPA target threshold at 175m ($250m \times 0.70$) and trigger new replicas, similar to the earlier scenarios. However, after

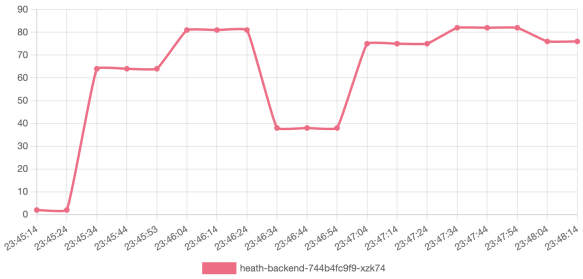

Figure 9: Backend pods CPU usage.

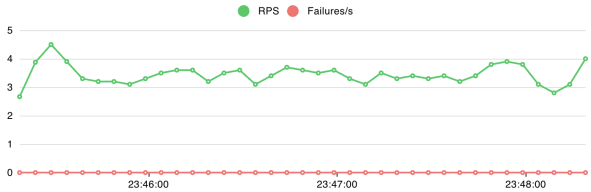

Figure 10: Locust RPS and failure rate.

increasing the limit to 3000m, the CPU usage never reached the 175m level and HPA was not triggered (see Figure 9). We thought it might be random, so we repeated the test three more times, but the result was the same each time. We investigated but could not find a clear reason for this behavior. In this run, the current replica handled the load comfortably, and since we did not wait for new replicas to start up, we did not observe any failures (Figure 10).

4.2 Database & Compute Bottlenecks

In this phase, we investigated the impact of the Database Virtual Machine (VM) size on the overall system stability. We compared two distinct configurations (Test 2.2 and Test 2.3) to isolate the hardware bottlenecks.

4.2.1 Undersized Database VM (e2-medium)

Configuration:

- **Database VM:** e2-medium (2 vCPUs, 4GB RAM)
- **Cluster Nodes:** 1 Node (e2-standard-2)
- **Load:** 2000 Users

Analysis: In this initial high-load test, the system experienced catastrophic failure. The underlying infrastructure metrics revealed a clear hardware bottleneck, despite the GKE cluster itself having remaining capacity.

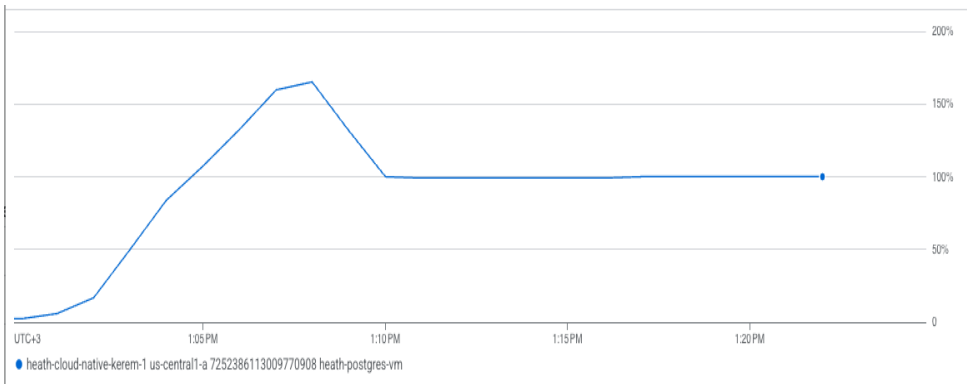

Figure 11: VM CPU Utilization: Reaching 100% Saturation

The **VM CPU Utilization** graph confirms that the 2 vCPUs of the `e2-medium` instance were completely saturated.

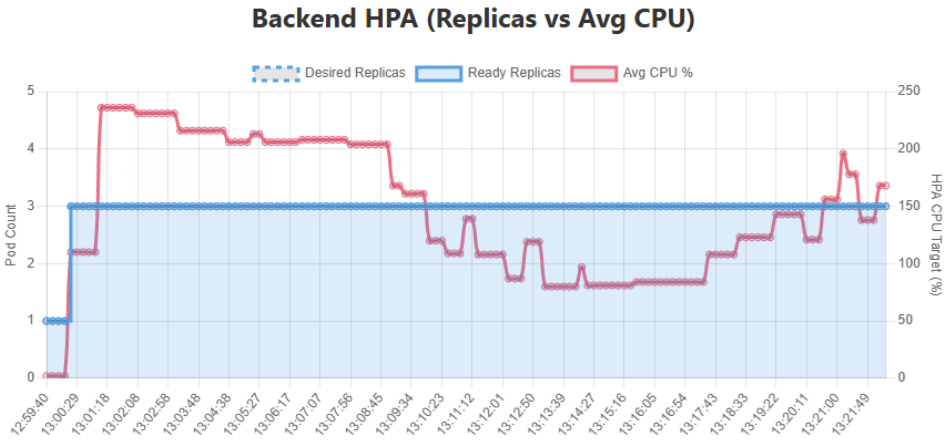

Figure 12: Backend HPA: Replicas failing to stabilize

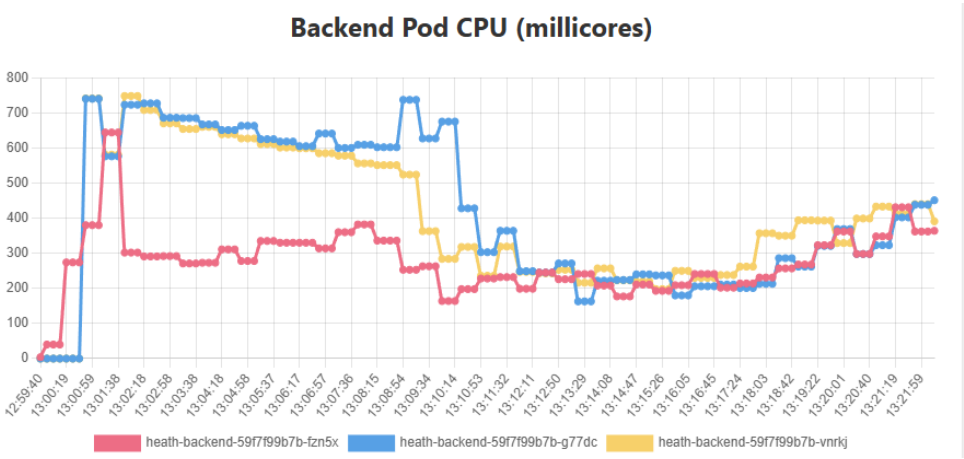

Figure 13: Backend Pod CPU: Erratic behavior due to DB Latency.

As shown in the graphs above, the GKE node itself had available CPU headroom. The **Backend HPA** graph shows the Average CPU percentage hovering around **150%**. Since our CPU Request was set to `250m`, this 150% utilization corresponds to approximately **375m** per pod. With 3 replicas running, the total backend consumption was roughly **1125m** (1.1 vCPUs).

Given that our `e2-standard-2` node provides **2000m** (2 vCPUs) of allocatable capacity, the cluster was only using about **60%** of its total power. This confirms that the Kubernetes cluster was not the limiting factor. Instead, the pods were forced to wait for query responses from the saturated database, leading to timeouts and preventing them from processing enough requests to utilize the full node capacity.

4.2.2 Oversized Database, Undersized Cluster

Configuration:

- **Database VM:** Upgraded to **e2-standard-8** (8 vCPUs, 32GB RAM)
- **Cluster Nodes:** 1 Node (e2-standard-2)
- **Load:** 2000 Users

Analysis: To resolve the database bottleneck, we vertically scaled the DB VM significantly. While this successfully prevented the database itself from crashing (keeping VM CPU utilization low), it shifted the pressure entirely to the GKE Cluster.

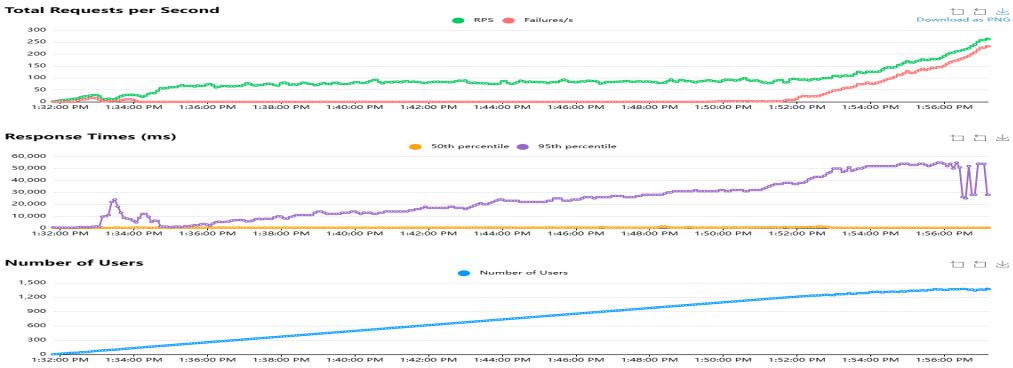

Figure 14: Failures persist due to cluster resource limits.

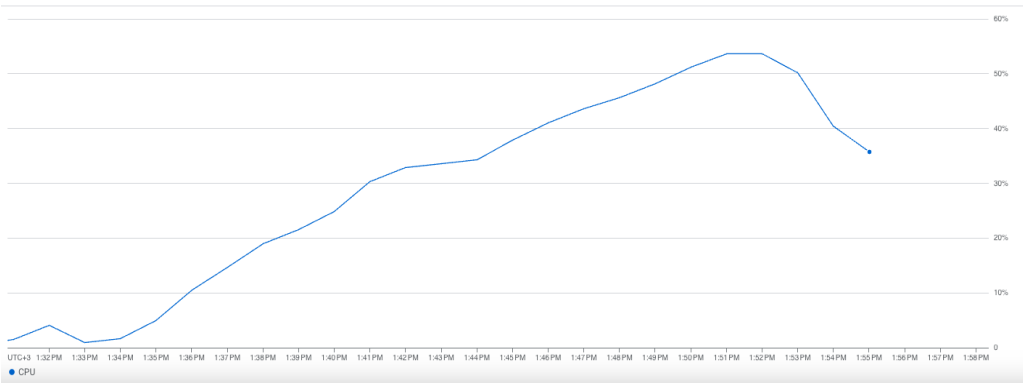

Figure 15: Database VM CPU: Healthy (under 40%) due to vertical scaling.

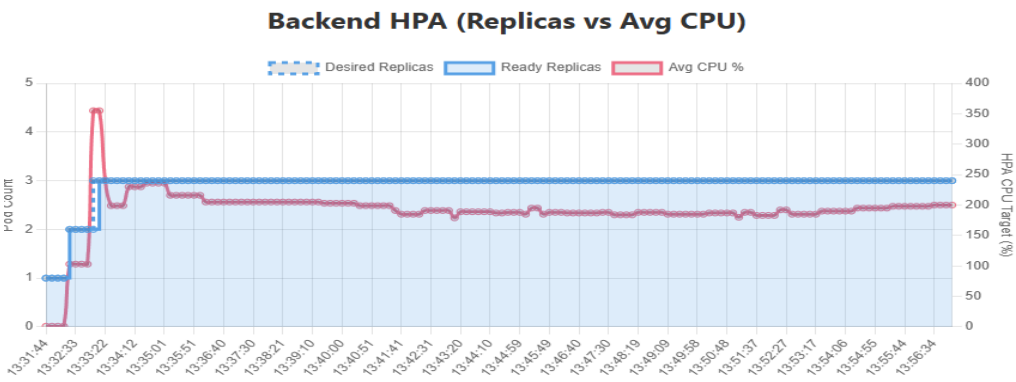

Figure 16: Backend HPA: Replicas fighting for Node CPU.

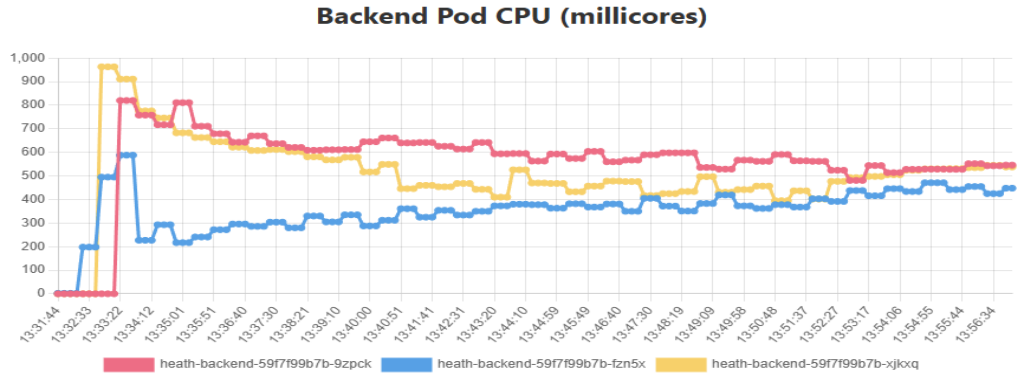

Figure 17: Backend Pod CPU: Node saturation preventing effective scaling.

Observation: The graphs reveal a classic "shifting bottleneck." By fixing the database capacity, we allowed more requests to reach the backend application. However, the single `e2-standard-2` Kubernetes node (with only 2 vCPUs) was immediately overwhelmed by the CPU-intensive authentication tasks of around 1300 concurrent users. The node hit 100% utilization, causing pod starvation and system-wide failure, proving that we needed to scale the GKE cluster (add more nodes) in addition to the database.

4.2.3 Cluster Autoscaling & Connection Exhaustion

Configuration:

- **Database VM:** e2-standard-8 (8 vCPUs)
- **Cluster Nodes:** Autoscaling Enabled (1 → 3 Nodes)
- **Max Replicas:** Increased to **9** (to force node scaling)
- **Load:** 2000 Users

Analysis: In this test, we successfully triggered the GKE Cluster Autoscaler. By increasing the HPA max replicas to 9, the system requested more CPU than a single node could provide, forcing GKE to provision two additional nodes. While this solved the compute bottleneck, it exposed the final limit of the default configuration: **Database Connection Pool Exhaustion**.

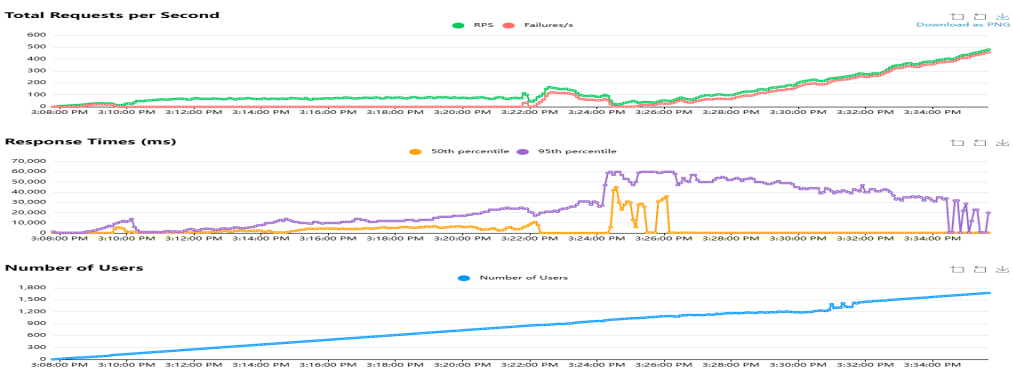

Figure 18: Failures persist despite adding nodes.

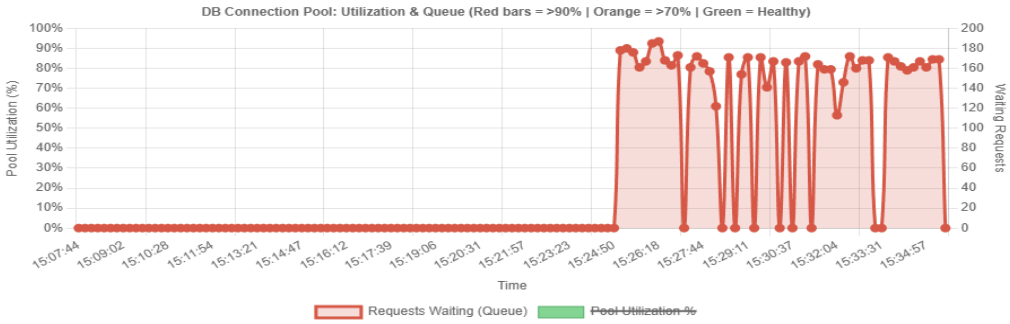

Figure 19: DB Connection Pool: High Waiting Requests (Queueing) causing the failures.

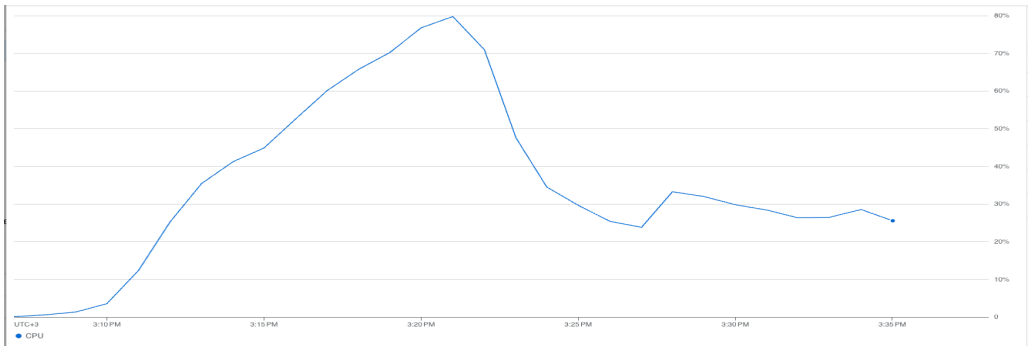

Figure 20: Database VM CPU low utilization, confirming compute is not the issue.

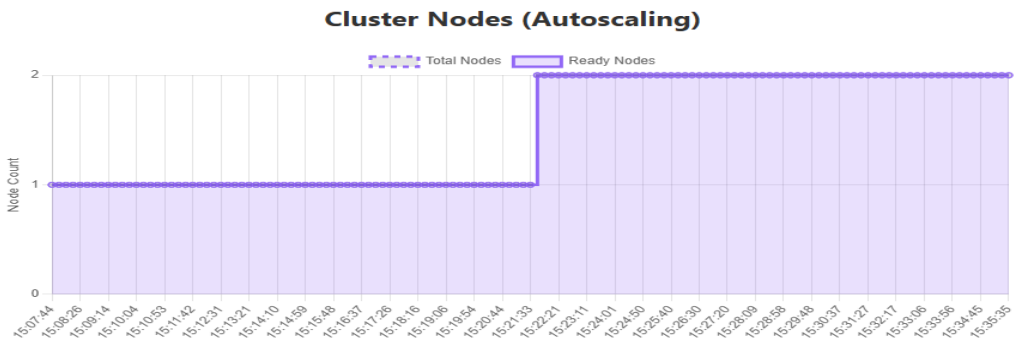

Figure 21: Cluster Autoscaler: Scaling from 1 to 2 Nodes (Infrastructure is working).

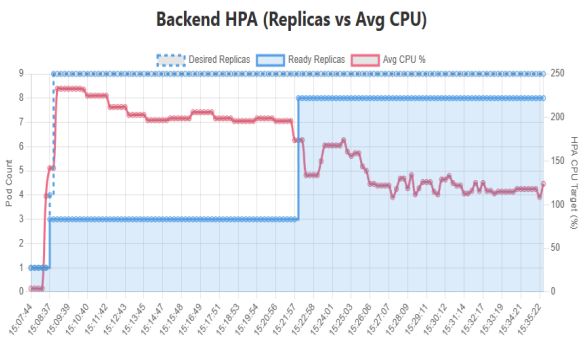

Figure 22: Backend HPA: Replicas -¿ 9.

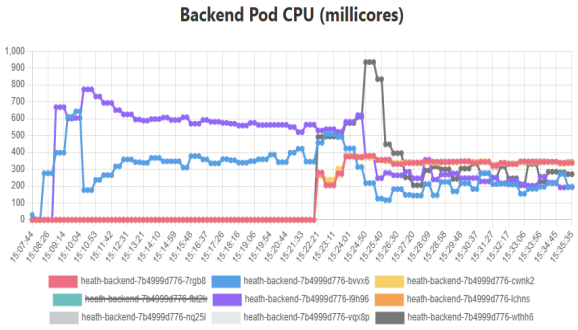

Figure 23: Backend Pod CPU: Usage drops.

Observation: The graphs demonstrate successful infrastructure scaling, yet failure persisted due to configuration limits:

- **Successful Scaling:** The **Backend HPA** correctly scaled replicas to 9, and the **Cluster Autoscaler** (Figure 21) successfully provisioned two additional nodes to accommodate them.
- **The Real Bottleneck:** The **DB Connection Pool** graph (Figure 19) reveals the true culprit. The "Waiting Requests" count spiked dramatically, indicating that application threads were blocked waiting for a database connection.
- **Root Cause:** The default HikariCP pool size (10) and PostgreSQL max connections (100) were insufficient for 9 replicas handling 2000 concurrent users. This resource exhaustion caused the timeouts and failures seen in the Locust chart, despite the abundance of available CPU and RAM.

4.3 Connection Pool Optimization

In this phase, we iteratively tuned the database connection settings and pod resource allocations to eliminate the remaining bottlenecks. Some adjustments were made *mid-test* using `kubectl` commands to observe real-time effects.

Configuration Summary (from left to right):

- **Test 1: Application-Side Pool Fix (Partial Success)**
  - **Change:** HikariCP pool size: 10 → 50 (PostgreSQL `max_connections` stayed 100).
  - **Result:** Reduced application-level waiting.
  - **New bottleneck:** PostgreSQL `max_connections` (100) became the hard limit.
  - **Observation:** VM CPU stayed low, so hardware was not the bottleneck.

- **Test 2: Database-Side Fix (New Issues Emerge)**
  - **Change:** PostgreSQL `max_connections`: 100 → 1000; DB right-sized to 6 vCPU.
  - **Autoscaling behavior:** Cluster Autoscaler triggered (1 → 2 nodes), causing pod redistribution.
  - **Mid-test intervention (16:43):** Manually deleted pods on a specific node to force redistribution.
  - **Issue:** Simultaneous pod restarts overwhelmed the pool → large "Requests Waiting" spikes.
  - **Impact:** 95th percentile response times increased to 30–60s.

- **Test 3: Resource Tuning (Best Scenario)**
  - **Change:** Backend CPU request: 250m → 500m.
  - **Mid-test interventions:**
    - Rolling restart at 17:46 to rebalance load.

    * Updated CPU requests live at 17:52 (`--requests=cpu=500m`).
  - **Result:** System ran at near-full utilization (VM, Node, Pod CPU) with minimal DB pool waiting.
  - **Limit:** Still could not sustain 2000 users without failures.

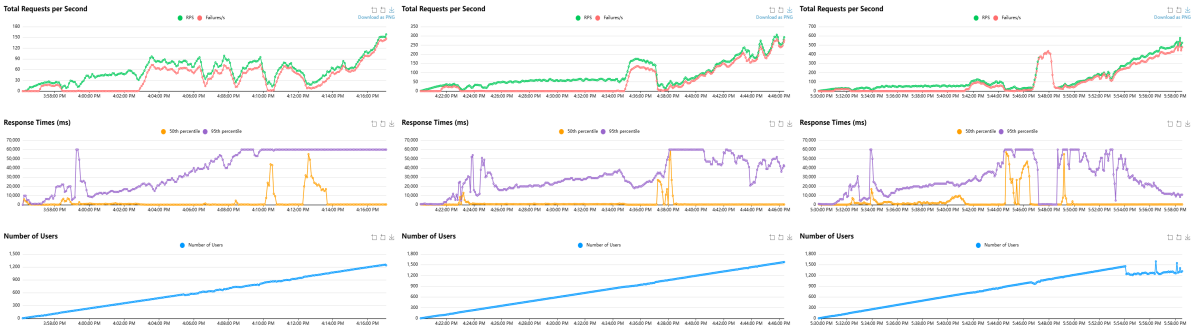

Figure 24: Locust Statistics: Test 1 (left), Test 2 (center), Test 3 (right)

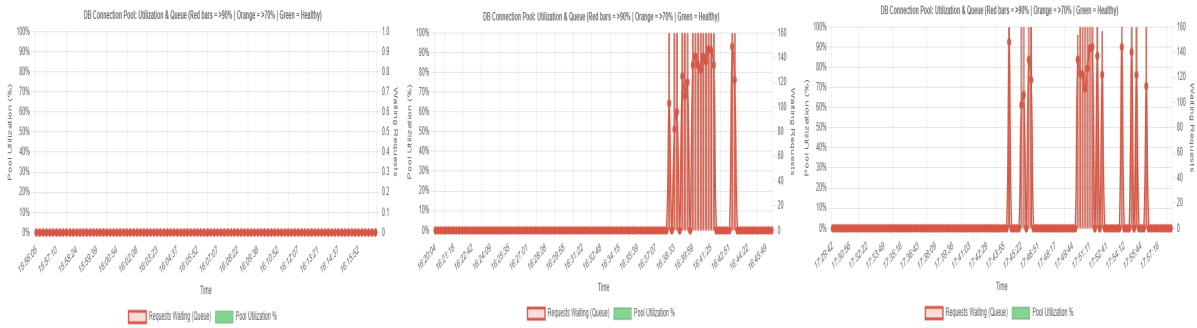

Figure 25: DB Connection Pool: Test 1 (left), Test 2 (center), Test 3 (right)

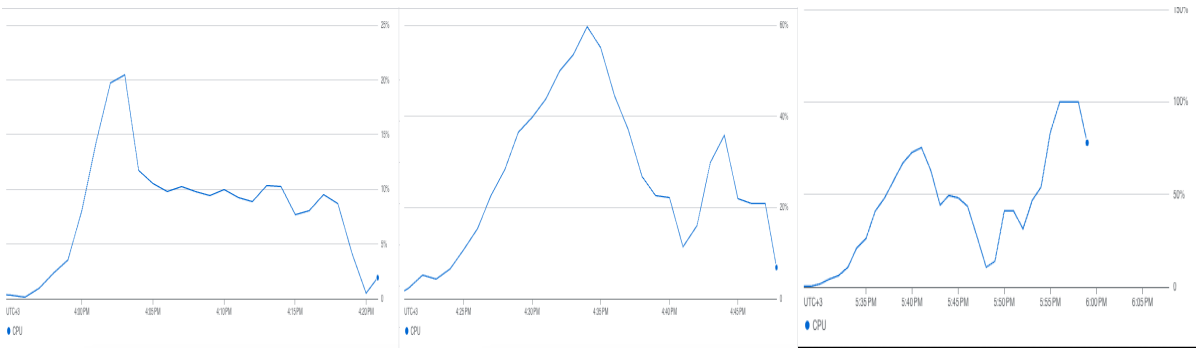

Figure 26: Database VM CPU Utilization: Test 1 (left), Test 2 (center), Test 3 (right)

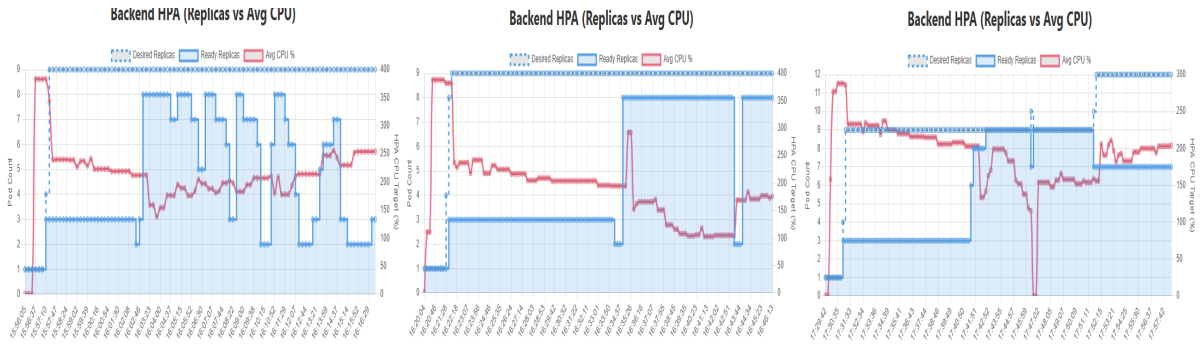

Figure 27: Backend HPA Replicas: Test 1 (left), Test 2 (center), Test 3 (right)

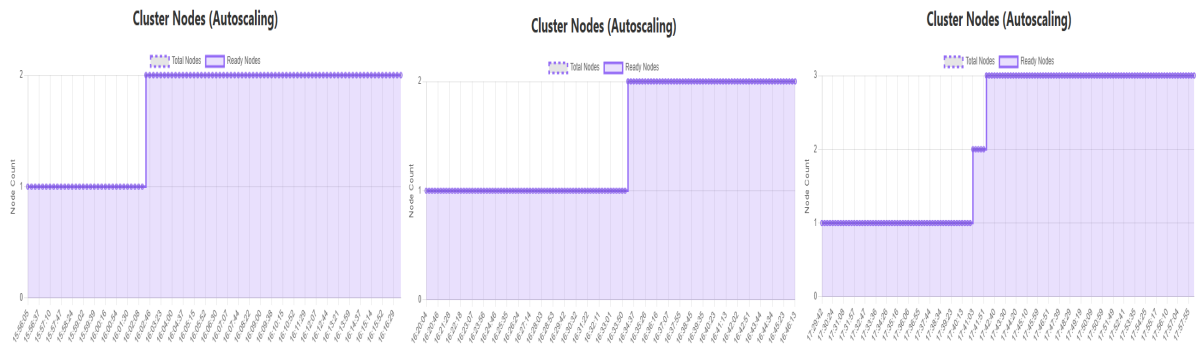

Figure 28: Cluster Node Count: Test 1 (left), Test 2 (center), Test 3 (right)

Conclusion: Test 3 represents the **best achievable scenario** with the current infrastructure. All artificial bottlenecks (connection pools, CPU limits) have been eliminated and the system now operates at full capacity. However, **2000 users exceeds the physical limits** of a 2-node cluster with e2-standard-2 machines. To support this load, the infrastructure must be scaled: increase `gke_max_node_count` or use larger nodes (`e2-standard-4`).

4.4 Node Pre-Provisioning: Cold vs Warm Start

This phase investigates the impact of the **initial node count** on system stability during traffic ramp-up. We compared two scenarios: starting with a single node (cold-start) versus pre-provisioning the maximum node capacity (warm-start). Both tests used identical configurations except for the starting node count.

Shared Configuration:

- **Database VM:** e2-custom-6-24576 (6 vCPUs, 24 GB RAM)
- **GKE Nodes:** e2-standard-2, Autoscaling Enabled (max 3 nodes)
- **Backend:** CPU request 500m, limit 1000m, HPA 1→9 replicas, 70% target
- **DB Pool Size:** 50 connections per pod
- **Load:** 1000 Users, Spawn Rate 20 users/sec

Test N1 (Cold Start):

- `gke_min_node_count = 1` — Cluster starts with 1 node.
- Test started at **1:00:00 AM**.
- Initial failure spike and 95th percentile response times hit **60,000ms** at 1:01:00–1:02:00 AM as the single node saturated.
- The "Gap of Death" extended from **1:01:00 to 1:08:00 AM**—multiple severe response time spikes (60,000ms) occurred at 1:02:00, 1:03:00–1:04:00, 1:05:00, and 1:07:00–1:08:00 AM as new nodes were provisioned one by one.
- RPS fluctuated heavily between 20–60 during instability, then spiked to **150 RPS** at 1:08:00 AM as backlogged requests cleared.
- System stabilized at **1:09:00 AM** with consistent 60–80 RPS and near-zero response times.
- **Time to stability: ∼9 minutes.**

Test N3 (Warm Start):

- `gke_min_node_count = 3` — Cluster pre-provisions 3 nodes.
- Test started at **1:40:00 AM**.
- Brief initial spike at **1:41:00–1:42:00 AM** during user ramp-up (expected JVM warm-up).
- Response times dropped sharply by **1:42:00 AM**—50th percentile stabilized almost immediately.
- Minor disruption at **1:45:00–1:46:00 AM** (HPA scaling event), recovered quickly.
- Full stability achieved by **1:47:00 AM** with consistent 60–80 RPS and response times under 10,000ms.
- **Time to stability: ∼7 minutes** (with significantly fewer and shorter disruptions).

Sweet Spot Configuration: Test N3 represents the optimal configuration for this workload:

- Pre-provisioned 3-node cluster (e2-standard-2)
- Right-sized database (6 vCPU, 24 GB RAM)
- Tuned connection pool (50 per pod)
- HPA: 500m CPU request, 70% target, max 9 replicas

This configuration successfully handles **1000 concurrent users** with stable response times, eliminating the prolonged "Gap of Death" seen in cold-start scenarios.

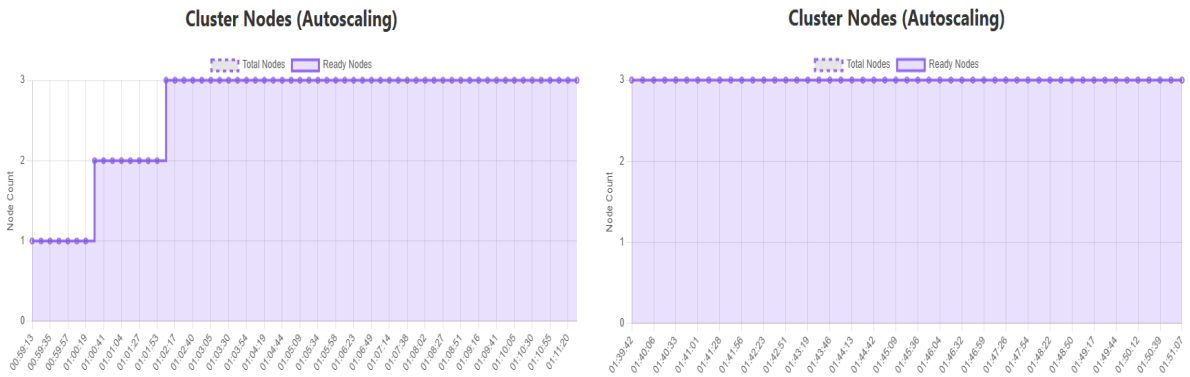

Figure 29: Cluster Node Count: Test N1 (left), Test N3 (right)

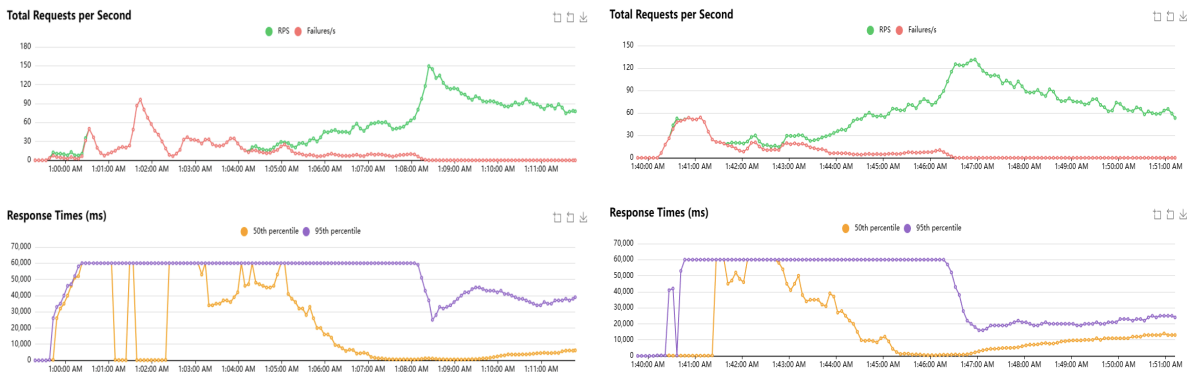

Figure 30: Locust Statistics: Test N1 (left), Test N3 (right)

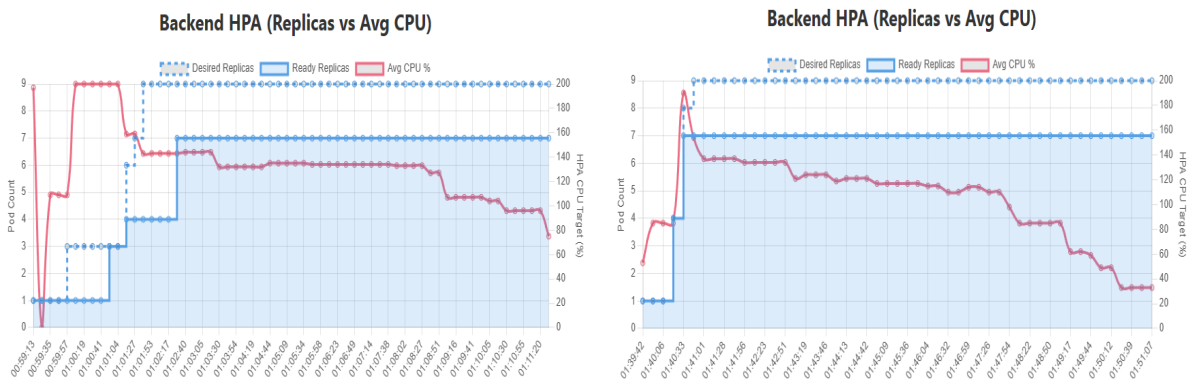

Figure 31: Backend HPA Replicas: Test N1 (left), Test N3 (right)

Figure 32: Backend Pod CPU (Millicores): Test N1 (left), Test N3 (right)

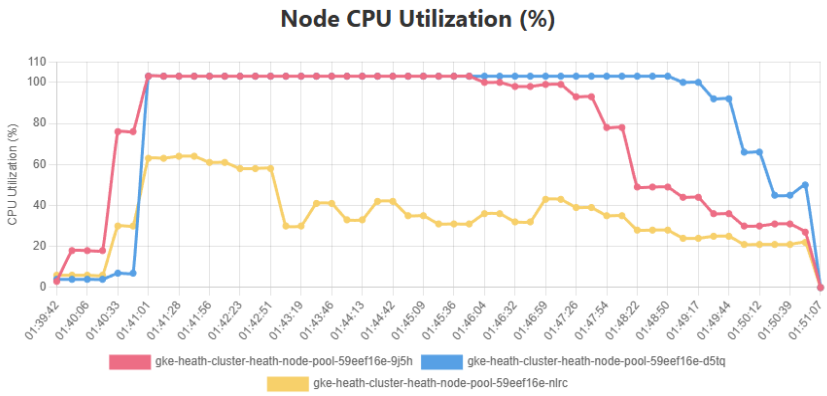

Figure 33: Test N3 Node CPU Utilization

Conclusion: The cold-start test experienced **7 minutes of instability** with multiple 60-second response time spikes, while the warm-start test had only **2 minutes of brief disruption**. Pre-provisioning nodes eliminates cascading scaling delays and is recommended for production workloads.

5 Cost Analysis

Table 2 details the estimated monthly costs for running the application infrastructure on GCP. The analysis assumes a baseline load with a single GKE node and a standard database instance size.

Category	**Component**	**Monthly Cost (Est.)**
Compute	GKE Cluster Management Fee	$73.00
	GKE Worker Node (1x e2-standard-2)	$49.00
	Database VM (e2-medium)	$25.00
Network	Load Balancer Forwarding Rule	$18.00
	Static External IP	$2.90
Storage	Artifact Registry	~$0.10
Total		**~$168.00**

Table 2: Estimated cloud infrastructure costs.

The total estimated monthly cost is approximately **$168.00**. This estimate accounts for the GKE control plane management fee, the compute resources for the worker nodes and database, and networking components required for external access. Costs may vary based on actual autoscaling behavior (if node count increases) and data transfer volume.